

What is Apache Airflow?

Apache Airflow is an open-source workflow orchestration platform used to automate, schedule, manage, and monitor complex workflows and data pipelines.

It was originally developed by Airbnb and later became an Apache Software Foundation project.

Airflow is mainly used in:

- Data Engineering
- ETL Pipelines
- Machine Learning Pipelines
- Cloud Data Processing
- Batch Processing
- Analytics Workflows

In Airflow, workflows are written using Python code, which makes it flexible and easy for developers.

The main idea of Airflow is:

“Define workflows as code.”

This means developers can create, schedule, test, and monitor workflows programmatically instead of manually running tasks.

Advantages of Airflow

1. Open Source and Free

Airflow is completely open source, so organizations can use it without licensing cost. It also has a huge developer community and strong documentation support.

2. Python-Based Platform

Since workflows are written in Python, developers can easily create dynamic and customizable pipelines.

Example:

- Creating loops
- Dynamic task generation

- Conditional workflows

This makes Airflow highly flexible compared to traditional scheduling tools.

3. Powerful Scheduling System

Airflow can schedule workflows:

- Hourly
- Daily
- Weekly
- Monthly
- Custom intervals

It also supports cron expressions for advanced scheduling.

Example:

- Run ETL every midnight
 - Generate reports every Monday
 - Trigger ML training weekly
-

4. Monitoring and Visualization

Airflow provides a web-based UI where users can:

- View DAGs
- Check task status
- Monitor execution time
- Restart failed tasks
- View logs

This makes troubleshooting easier.

5. Scalability

Airflow supports distributed execution using multiple workers and executors.

It can handle:

- Small workflows
- Enterprise-level pipelines

- Thousands of tasks

Large companies use Airflow for big data processing.

6. Fault Tolerance and Retry Mechanism

If a task fails:

- Airflow can retry automatically
- Alerts can be configured
- Failed tasks can restart from checkpoints

This increases reliability.

7. Integration Support

Airflow integrates with many technologies such as:

- Apache Spark
- Hadoop
- Hive
- AWS
- Azure
- Google Cloud
- Databricks
- SQL Databases
- Snowflake

Because of this, it is widely used in cloud and big data environments.

8. Dynamic Workflow Creation

Airflow allows workflows to change dynamically based on conditions and data availability.

Example:

- Execute task only if file exists
 - Run pipeline based on API response
-

Airflow Architecture

Airflow architecture consists of multiple components working together to execute workflows.

1. Scheduler

The Scheduler is the heart of Airflow.

Responsibilities:

- Monitors DAGs
- Checks scheduling time
- Triggers tasks
- Sends tasks to executors

It continuously scans workflows and decides which tasks should run.

2. Web Server

The Web Server provides the Airflow User Interface (UI).

Using UI, users can:

- Monitor DAG execution
- Trigger workflows manually
- Check logs
- Pause/resume DAGs
- Track failures

It provides easy visualization of workflows.

3. Metadata Database

Airflow stores all metadata in a database.

Stores:

- DAG information
- Task states
- Logs
- User details

- Scheduling history

Common databases:

- PostgreSQL
 - MySQL
 - SQLite (for testing)
-

4. Executor

The Executor decides how tasks are executed.

It receives tasks from the Scheduler and assigns them for execution.

Different executors are used depending on workload and scalability needs.

5. Workers

Workers are machines or processes that actually execute tasks.

In distributed systems:

- Multiple workers run tasks simultaneously
 - Improves performance
 - Supports parallel execution
-

6. DAG Folder

This folder stores all DAG Python files.

The scheduler continuously scans this folder to detect workflows.

What is a DAG?

A **DAG (Directed Acyclic Graph)** is the core concept in Airflow.

A DAG represents a workflow made up of tasks connected in a specific order.

Meaning of DAG

Directed

Tasks follow a defined direction.

Example:

Task A → Task B → Task C

Acyclic

There should be no loops.

Correct:

A → B → C

Wrong:

A → B → C → A

Graph

A collection of connected tasks.

Features of DAG

- Defines workflow structure
 - Maintains task dependencies
 - Supports parallel execution
 - Supports scheduling
 - Helps monitor workflows easily
-

Example of DAG Workflow

Example ETL Pipeline:

Extract Data → Transform Data → Load Data

Here:

- Transformation starts only after extraction completes

- Loading starts only after transformation completes
-

What are Executors?

Executors are components responsible for running tasks in Airflow.

The Scheduler sends tasks to the Executor, and the Executor decides:

- Where task runs
 - How task runs
 - Whether tasks run sequentially or parallelly
-

Types of Executors

1. Sequential Executor

- Default executor
- Runs one task at a time
- Mainly used for testing and learning

Advantages:

- Simple setup

Disadvantages:

- No parallel processing
 - Slow for production
-

2. Local Executor

Runs multiple tasks parallelly on the same machine.

Advantages:

- Faster than Sequential Executor
- Supports parallel execution

Disadvantages:

- Limited by machine resources

3. Celery Executor

Uses distributed workers for task execution.

Advantages:

- Highly scalable
- Supports distributed systems
- Good for large organizations

Disadvantages:

- Complex setup
 - Requires message broker like Redis or RabbitMQ
-

4. Kubernetes Executor

Runs each task in a separate Kubernetes pod.

Advantages:

- Highly scalable
- Better resource isolation
- Cloud-native execution

Disadvantages:

- Requires Kubernetes knowledge
-

Difference Between DAG and Executor

DAG	Executor
Defines workflow structure	Executes tasks
Represents dependencies	Controls execution
Written in Python	Part of Airflow architecture
Logical workflow	Runtime execution component

Real-Time Example of Airflow

Suppose a company wants to generate a sales report daily.

Workflow:

1. Extract sales data from database
2. Transform data
3. Load into data warehouse
4. Generate dashboard
5. Send email report

Airflow automates this complete process daily without manual work.